Chapter 4 解答

CH04-Q01

1. `lw` path 的總和是

$$30 + 200 + 120 + 180 + 250 + 20 + 40 = 840 \text{ ps}.$$

簡化 clock period 至少 840 ps。真實驗證還要加 skew、jitter、wire delay 與 hold constraint。

2. R-type 不經 data memory 與 memory-to-register mux，因此 logic path 比 `lw` 少

$$250 + 20 = 270 \text{ ps}.$$

其 path 是 570 ps，但 clock 仍由 840 ps 的 `lw` 決定。

3. 程式中的 R-type 只占部分 instructions。Single-cycle 每次 instruction 都使用同一個 840 ps clock，故縮短 R-type path 沒有縮短 clock period。最長path仍是lw的840 ps，R-type由570降至470 ps不改變這個最大值；只有降低限制時脈的最慢path，且沒有其他path形成同樣或更大的限制，才可縮短共同週期。

4. 會。若 `lw` 仍是全體最長 path，clock period 可降至 760 ps；所有 instructions 都在同一個新 clock 下執行。收益代價是 data-memory 的更快 implementation 可能增加面積、power 或降低容量。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

CH04-Q02

1. 時空圖如下。

Instr	IF	ID	EX	MEM	WB
I1	1	2	3	4	5
I2	2	3	4	5	6
I3	3	4	5	6	7
I4	4	5	6	7	8
I5	5	6	7	8	9
I6	6	7	8	9	10
I7	7	8	9	10	11

2. 最後一條 instruction I7 的 WB cycle 是 11。
3. 所需cycles為 $5 + (7 - 1) = 11$，有限批次的平均IPC為 $7/11 \approx 0.636$。題目明定pipeline每cycle200 ps，所以時間為2200 ps；未管線化每條1000 ps，7條需7000 ps。相同工作的時間speedup為 $7000/2200 = 35/11 \approx 3.1818$。不能只除兩邊cycle數，因為clock periods不同；忽略startup的steady-state IPC=1也不是此7條批次的平均IPC。
4. 單筆 latency 是 $5 \times 200 = 1000$ ps。完成全部 7 條是 $11 \times 200 = 2200$ ps。Steady-state 每 200 ps 完成一條，不等於單筆只花 200 ps。

CH04-Q03

1. 無 stall 的基本圖。

Instr	IF	ID	EX	MEM	WB
I1	1	2	3	4	5
I2	2	3	4	5	6
I3	3	4	5	6	7

2. I1 的 `$8` 在 cycle 4 位於 EX/MEM，I2 在 cycle 4 的 EX 需要它，所以從 EX/MEM 前送。
3. I3 在 cycle 5 位於 EX，`$8` 在 cycle 5 位於 MEM/WB，可從 MEM/WB 前送；I3 的 `$11` 是 I2 在 cycle 5 的 EX/MEM result，從 EX/MEM 前送。兩條 paths 的 source selection 不能混為一談。
4. 若漏掉前送，I2 可能讀到 `$8` 在 I1 WB 前的舊 register-file 值。若只有 I1→I2 的前送缺失，而其他前送正常，I3 使用由錯誤輸入算出的 `$11`，但其 `$8` 可由 I1 的 MEM/WB 正確前送；其他路徑也缺失時才可能讀到舊 `$8`。

CH04-Q04

1. ID/EX 的 producer 是 `lw`，有 `MemRead=1`，destination `rt=$2`；IF/ID 的 consumer `and` reads `$2` 作 `rs`。所以 load-use hazard 成立，需要一個 stall。
2. PC 保持原值，IF/ID 保持 I2，ID/EX 插入 control bubble/no-op。I1 繼續到 MEM；下一 cycle loaded data 可由 MEM/WB 前送到 I2 的 EX。
3. 時空圖：

Instr	IF	ID	EX	MEM	WB
I1 lw	1	2	3	4	5
I2 and	2	3、4（停留）	5	6	7
bubble			4	5	6
I3 or	3、4（取指保持）	5	6	7	8

cycle 4 的 EX 是 bubble，不能同時放 I2。I2 在cycle 5的EX使用MEM/WB前送的load data。I3的IF/ID寫入在停頓時被禁止，延至cycle 5才進ID。 4. `sw` 使用 `rt` 作為 value source，`lw` 的 `rt` 是 destination。硬體 hazard check 必須確認 waiting instruction 是否真的讀該 `rt`；若 `sw` 僅以 `$2` 作 store data，具 MEM/WB→MEM bypass 的設計可不 stall；若必須在 EX 前取得 store data 則需 stall。若 base 本身依賴 load，仍需依 EX 使用時點處理。

CH04-Q05

初始 WN。預測 not taken。

Step	Outcome	Prediction	Result	New state
1	T	N	miss	WT
2	T	T	hit	ST
3	N	T	miss	WT
4	T	T	hit	ST
5	N	T	miss	WT
6	N	T	miss	WN
7	N	N	hit	SN

1. 狀態與結果如表。
2. Mispredictions 4 次，accuracy $3/7 \approx 42.9\%$。
3. 依題目明定的兩個slot恢復成本，四次方向誤判各浪費2個wrong-path slots，共8個。預測taken但實際not-taken同樣需要丟棄預測路徑，不能只計實際taken的誤判。
4. 單筆 branch accuracy 不含 resolve stage、wrong-path instructions 數量、BTB miss、target alias、不同 branch types 與 recovery cost。

CH04-Q06

1. 保留較舊且無 exception 的 `I2`、`I3` architectural effects；`I2` 在 WB 的 commit 與 `I3` 在下一 cycle 的 WB 都要完成。`I1` 已在先前 cycle commit。
2. 阻止 faulting `I4` 與較年輕的 `I5`、`I6` 寫 register 或 memory。`I5` 在 ID、`I6` 在 IF，兩者都尚未 commit。
3. 保存 `ExceptionPC=I4.PC`、overflow cause、必要的 bad address 與 privilege/status context。`I3` 在 `T+1` 完成 WB/commit 後，handler 才能看到等於依序執行 `I1`、`I2`、`I3` 的 state；hardware 在 `T+2` redirect。
4. ROB 讓亂序結果先暫存。當 `I4` 成為 ROB head 前，必須先依序 commit `I2`、`I3`；之後丟棄 `I4` 與所有較年輕 entries，恢復 rename mapping，再 redirect。年輕 instructions 可以執行，但不得先改變 architectural state。

CH04-Q07

1. 基本 condition 是

```
ID/EX.MemRead
AND ((IF/ID.uses_rs AND ID/EX.RegisterRt == IF/ID.RegisterRs)
     OR (IF/ID.uses_rt AND ID/EX.RegisterRt == IF/ID.RegisterRt))
AND destination is not $zero
```

兩側均須確認來源實際使用，且本條件假設該值在下個 EX 需要；若 store-data 到 MEM 才使用，另依 bypass 時序處理。 2. `$zero` 的 value 永遠是 0，對它的 write 被忽略，不能把它當成 producer；若產生錯誤 stall 會降低效能但不一定影響 correctness。 3. 用 IF/ID 的 `RegRtUsed` 或 source-use control signal，只在 consumer 真的讀 `rt` 時成立。`lw`/`sw` 的 `rt` semantics 不同，需依 instruction class 解釋。 4. 同一個 register 若同時可由 EX/MEM 與 MEM/WB 提供，選可用的最新 EX/MEM result；若最新 producer 是 load，其 EX/MEM 值只是位址，尚不可當載入值，須先 stall 或等可用前送。較舊 MEM/WB value 可能已被後續 instruction 覆寫；forwarding 的目的就是維持 program-order newest-value semantics。

CH04-Q08

題目給的是cycles比例，不是instructions比例。以cycle為共同分母，

$$IPC = 0.60(4) + 0.30(2) + 0.10(1) = 3.10.$$

1. 理想平均所需cycles為 $10000/3.10 \approx 3225.81$。有限實際執行以整數cycle完成，精確比例未必可同時成立，本題採加權平均近似。不能以 $0.6(10000)/4$ 計算，因為那把cycle比例誤當instruction比例。
2. 時間為 $3225.81/(2 \times 10^9) \approx 1.61290\ \mu s$。
3. 雙發射需 $10000/2 = 5000$ cycles，在2.4 GHz下為 $2.08333\ \mu s$。四發射模型較快；其speedup約 $2.08333/1.61290 = 1.29167$。
4. 已知issue分布只描述本題假設的指令供應。branch誤判、cache miss、register-port conflict與thermal throttling若尚未納入該分布，就必須另建模，不能由峰值或這個加權平均自動推得。

CH04-Q09

1. Weighted CPI：
$$\text{CPI} = 0.45(4) + 0.20(3) + 0.20(5) + 0.15(4) = 4.0.$$
2. 10^9 instructions 的時間：
$$T = 10^9 \times 4.0 \times 300\,\text{ps} = 1.2\,\text{s}.$$
3. Single-cycle 時間是 $10^9 \times 1 \times 800\,\text{ps} = 0.8\,\text{s}$，所以此題的 single-cycle 較快。多週期可用較短 cycle 不代表 CPI 增加後仍會勝出。
4. 令最大 cycle time 為 t：
$$10^9(4.0)t = 0.8\,\text{s} \quad \Rightarrow \quad t = 200\,\text{ps}.$$

若每 cycle 仍大於 200 ps，多週期設計較慢。此反解未改變 state path、CPI 或 instruction mix。

CH04-Q10

1.
$$\text{mispredict_cycles} = 0.20 \times 0.60 \times 2 = 0.24.$$
2. Load-use stall term 是 0.10，因此
$$\text{CPI} = 1 + 0.24 + 0.10 = 1.34.$$
3. No-delay-slot 的 flush penalty 是丟棄兩個 fetched instructions。Classic MIPS delay slot 則把緊接 branch 的一條 instruction 視為必執行，flush 只作用於 slot 之後的 wrong path；slot 可由原本有用指令填入，也可能需新增 nop；動態指令數不一定增加，須依重排後程式重新計算。必須用同一份程式與完整 dynamic instruction count 比較，不能只把 penalty 2 改成 1。
4. `T` cycle 捕捉 `I4` 的 PC/cause（僅依例外類型記錄必要位址，overflow 不一定有 bad address），讓 `I2` commit，阻止 `I4/I5/I6` writes；`T+1` 讓 `I3` commit，再以 exception buffer 的資料在下一 cycle redirect 至 handler。Handler 看到的 architecture state 等於依序完成 `I1` 到 `I3`，`I4/I5/I6` 沒有任何 register 或 memory effects。

CH04-Q11

1. 第一個word的op=0、rs17、rt18、rd8、funct0x21，是addu t0,s1,s2。RegDst1、ALUSrc0、MemtoReg0、RegWrite1；ALU為7+5=12，WriteReg8，t0得到12，沒有data-memory寫入。

0x8e09000c是lw t1,12(s0)，op0x23、rs16、rt9、imm12。RegDst0、ALUSrc1、MemtoReg1、RegWrite1、MemRead1；ALU inputs0x1000與12，address0x100c，memory值0xabcdef01寫入t1。

0xae0afffc是sw t2,-4(s0)，op0x2b、rs16、rt10、imm0xfffc=-4。ALUSrc1、MemWrite1、RegWrite0，RegDst/MemtoReg的值不影響提交；address0x0ffc，RtData=0x12345678寫入該memory word。三者無branch/jump，所以各自PC=0x00400024。

2. Word解碼為beq s1,s2,-2。PC4=0x00400024，位移−8，BT=0x0040001c。7−5=2所以Zero0，nextPC=PC4；s1改5後5−5=0，Zero1且Branch1，nextPC=BT。兩種情況都不寫register或data memory。
3. PC4=(0x0ffffffc+4)=0x10000000，JT={1,26個0,00}=0x10000000。使用舊PC高4 bits會得到0，跨到錯誤256 MiB區域。這個邊界反例可辨識「高bits來自PC還是PC+4」的不同定義。
4. ALUSrc0把RtData當第二operand，address=0x1000+0x12345678=0x12346678，寫到錯地址；這是address計算輸入mux錯選。若address仍0x0ffc，但store data錯接signext immediate，會寫0xfffffffc而非0x12345678，這是Reg[rt]到memory write-data的分支接錯。ALUSrc只選ALU operand，不能同時取代store-data線。

CH04-Q12

有 forwarding：數字為 cycle 編號，多個數字表示保持同一 stage：

指令	IF	ID	EX	MEM	WB
I1	1	2	3	4	5
I2	2	3	4	5	6
I3	3	4, 5	6	7	8
I4	4, 5	6	7	8	9

C4 的 load-use 判斷使 C5.EX 為 bubble；較舊的 I1、I2 照常前進。C5 的 WB 已將 `$2=0x33` 寫入 register file，故 ID/EX 在 C5 結束可能保存 `0x33`。C6 的 MEM/WB 前送 `0x3c`，覆蓋這個舊值，算得 `$4=0x3c AND 0x0f=0x0c`。C7 再從 EX/MEM 前送 `0x0c`，算得 `$8=0x0c OR 0x80=0x8c`。完成時間為 C9。

完全無 forwarding 時：

指令	IF	ID	EX	MEM	WB
I1	1	2	3	4	5
I2	2	3	4	5	6
I3	3	4, 5, 6	7	8	9
I4	4, 5, 6	7, 8, 9	10	11	12

I3 等 I2 在 C6 寫回，才可於同 cycle 後半讀取；I4 等 I3 在 C9 寫回，才可於後半讀取。兩處各增加 2 個 cycles，總計 `4+4+2+2=12`。目的暫存器最後仍為 `$2=0x3c`、`$4=0x0c`、`$8=0x8c`，改變的是時間，不是程式結果。

第四小題中 I3 不再讀 `$2`，load-use 判斷為假。I3.EX 在 C5，以 `$10=0x55` 得 `$4=0x05`；I4.EX 在 C6 從 EX/MEM 前送 `0x05`，得 `$8=0x85`。四條指令無停頓，最後 WB 在 C8，共 8 cycles。I1、I2 雖都寫 `$2`，順序寫回即可維持正確的最終值，不能因此添加 RAW stall。

參考資料與署名

Lin Cing，《計算機組織》手寫筆記。 Patterson and Hennessy, Computer Organization and Design, revised fourth edition, Chapter 4. CSO course slides, processor datapaths and pipelines.

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。
https://github.com/xinhi6716/computer-science-self-study